

Stealthy and Practical Multi-modal Attacks on Mixed Reality Tracking

ANONYMOUS

Mixed Reality (MR) is rapidly becoming an essential technology for critical applications such as physical therapy and surgery, in addition to its early use for leisure activities. This transition necessitates a focused look at the security aspects of MR devices and applications. Prior work on MR security has primarily focused on generic security aspects such as secure authentication and vulnerability analysis. However, MR devices are multi-modal and spatiotemporal in nature, which exposes them to attacks on sensor modalities across spatiotemporal axes. Prior work has demonstrated attacks on individual sensing streams, but modern, state-of-the-art sensor fusion-based tracking algorithms can easily mitigate such attacks.

In this paper, we introduce a practical and stealthy attack surface; it concurrently launches attacks on multiple sensing streams across spatiotemporal axes to yield *fast*, *stealthy*, and *precise* outcomes. To the best of our knowledge, our work is the first to propose, design, and evaluate concurrent multi-modal spatiotemporal attacks. In doing so, we solve key challenges in deciding on what attack mechanisms to use, when to launch attacks, and how to configure attacks. Using the tracking and navigation use case of a user wearing a MR headset in real world settings, we demonstrate the effectiveness of our attacks over the user's trajectory in presence of state-of-the-art sensor fusion-based tracking algorithms and system checks.

1 INTRODUCTION

The emerging applications in interactive, mobile, wearable and ubiquitous technologies rely on multi-modal sensing and tracking of user activities and surrounding environment via commodity sensing devices such as Head Mounted Displays (HMD), hand-held controllers, and other devices [7, 32]. For example, immersive Mixed Reality (MR) systems center around macro- and micro-level tracking of humans and enhancing their experience by designing human-in-the-loop systems [55]. A key challenge for human-in-the-loop systems is the exploitation of security vulnerabilities that impact human safety [77]. Due to the prevalence of sensors on MR devices, their reduced field of view, and critical application use cases, MR devices can be a lucrative target for malicious activities with real-world consequences to human safety [39]. Recent studies on vulnerabilities in MR mostly investigate security and privacy issues around authentication [65], access control [54], and digital biomarkers [60]. However, these studies mainly focuses on high-level privacy implications of security threats and misses a discussion on the security of fundamental services such as tracking that pose threats to the physical safety of users.

The design of emerging applications in MR and related domains largely follows the *sense-compute-control* paradigm. The headset, or a wearable device, first senses user's motion and environment using visual and inertial sensors (*sense*). It then sends the data to the processing pipeline of a tracking algorithm that tracks user's motion and activities (*compute*). Typically, the tracking abilities of the headset are offered as a fundamental service, just like timing services, to the applications that run on the device. The novelty applications, such as the MR jogging application shown in Figure 1, leverage the tracking service of these devices to decide on an actionable feedback that controls user's action (*control*). There is significant work on developing accurate tracking services for the downstream applications using data from a wide range of sensing modalities. For example, prior work has developed sensor fusion algorithms that either use individual sensing modality or *fuse* data from multiple sensing modalities, such as visual and inertial sensors, to provide accurate and robust tracking services.

As these interactive and immersive applications become ubiquitous, there is a growing concern that a malicious entity can incur physical harm to the technology users who completely, or partially, rely on system feedback to complete various tasks. If an adversary successfully attacks the fundamental services such as tracking, all the downstream applications will be impacted. This impact will be further exacerbated if the malicious tracking is also stealthy i.e. not detected via system checks or through user observations. This concern stems from the existing or potential vulnerabilities at the *sense*, *compute*, and *control* stages of the application pipeline. For example, prior work has demonstrated successful attacks on individual sensing modalities such as inertial [67]

Author's address: Anonymous.

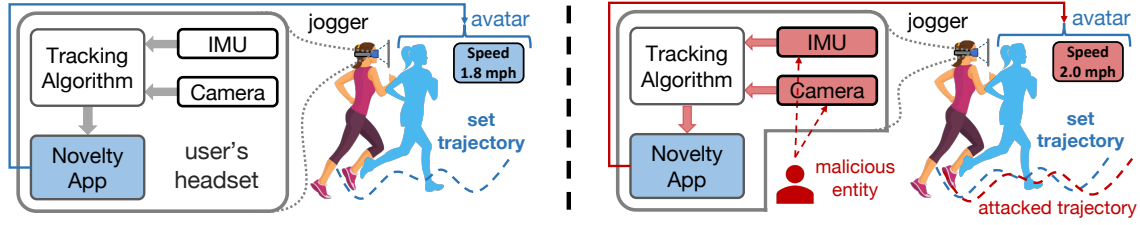


Fig. 1. Motivation scenario of our attacking approach. Left figure show a scenario where a user wearing a headset is being guided in the run by a novelty application through an avatar on a predefined trajectory. Right figure shows an attacker attacks the sensing streams, changing their output, and fooling the novelty app into changing the trajectory of the run and displaying incorrect speed. and visual [13]. These attacks have serious implications for the accuracy and security of the tracking services that rely on data from these sensing modalities. The recent work on leveraging sensor fusion, combining data from multiple sensors, have been successful in mitigating some of these threats. For instance, SelectFusion [15] is the state-of-the-art sensor fusion algorithm that successfully overcomes adverse effects on single sensor modality as shown in Figure 2(b), while traditional tracking algorithm ViNet [16] in Figure 2(a) fails to do so. The basic principle of SelectFusion tracking is that if data from one sensing modality degrades, the tracking algorithm prioritizes other modalities to mitigate the effects and offer accurate tracking.

While state-of-the-art sensor fusion-based tracking algorithm can mitigate the effect of attacks on individual sensing modalities, there are additional attack surfaces in MR tracking pipeline that have not been fully explored. To fool fusion algorithms, system checks, and human observations, we propose multi-modal attacks on MR tracking services. Our multi-modal attacks target multiple sensing streams simultaneously in a stealthy, precise, and practical manner. We show the impact of our multi-modal attack surface on human safety in MR through a jogging use case in Figure 1. In this case, a jogger runs alongside an avatar that is following a pre-defined trajectory and setting a fixed pace for the user. On the right, an attacker attacks both sensing modalities and fools the tracking service into believing that the user is running faster than the desired speed of 1.8mph. The application will reduce the pace of the avatar to ensure the user is not running faster, a feedback effect that will result in the user slowing down to a speed below 1.8mph. The attacker can similarly show that the user has deviated from her path and needs to change her course. Note that we do not assume any control or access to the novelty application or the sensor-fusion algorithm. There are numerous safety-critical MR applications in surgery, construction, and industrial control where malicious tracking from these attacks could be catastrophic.

The key property of multimodal learning techniques is that they effectively exploit the individual strengths of distinct sensors and coherently estimate the system states. For instance, inertial data streams have a strong temporal component, and are generally available at higher frequency than visual streams. They are mostly encoded via LSTMs [74], whereas visual streams have a stronger spatial component and are encoded via CNNs [17]. Spatial and temporal features extracted from different modalities are complementary, and thus, hints towards a common spatiotemporal attack surface for fusion protocols across applications, which is the focus of this paper. It is not trivial however to establish this new attack surface.

We overcome a variety of challenges to enable our multi-modal and spatiotemporal attack surface. First, it is challenging to craft adversarial examples to fool fusion algorithms through random perturbations for non-visual data such as inertial data due to their strict semantic constraints [62]. Though researchers have produced domain-specific adversarial attacks, they do not generalize beyond fixed application domains. To generalize across multimodal domains, we propose new domain object manipulations for sensor fusion applications that do not cause semantic inconsistencies. As most sensor data either has a strong temporal, spatial, or both components, we propose spatiotemporal frame-level manipulations, such as duplicating or dropping a frame for input perturbations. We argue that frame-level manipulations are simple, fast and practical. Additionally, to maintain stealthiness we develop new similarity index metrics that govern the perturbation threshold and evade system checks. Finally,

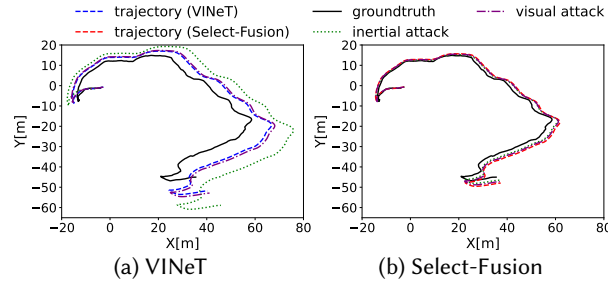


Fig. 2. State-of-the-art sensor fusion algorithm, Select-Fusion [15], is effective against attacks on a single sensing stream whereas simple tracking algorithm, VNet [16], fails.

our goal is to parameterize perturbations and allow the attacker to configure those parameters to ensure that certain configurations of attack always yield the same precise outcome. In essence, the goal of this attack surface is to launch *practical, stealthy and precise* attacks.

The proposed technical solutions to launching these attacks pose many low-level challenges that we solve using a systematic and iterative approach. First, while the frame-level manipulations, such as dropping or adding a frame, are straightforward for visual sensors like cameras, the notion of a frame does not exist for analog signals of inertial sensors. We define the notion of ‘frame’ for analog signals and design equivalent frame-level manipulation mechanisms for inertial sensors. Second, we analyze the similarity index values to investigate how it changes under attack in different environments. Our analysis guides our choice to attack sensing modalities only when they are in steadystate, i.e. the scene is not changing significantly. Third, our steadystate-based attacks are stealthy against systems checks like similarity index, they may still not be effective in achieving the desired change in user’s trajectory or speed. Our further analysis of the effects of controlled attacks suggests that attacks are only successful when the steadystate windows of different sensors overlap and both are attacked concurrently. Our controlled analysis to this point had assumed perfect future knowledge of user’s trajectory and its effects on the frame properties and similarity index. In practice, the attacker would not know when a steadystate window starts or ends. This leads to imprecise attacks that achieve in-deterministic deviations in user’s speed or trajectory. To solve this problem, we parameterize our attacks that are launched using our proposed algorithm called right frame selection (RFS) algorithm, which informs the attacker on when to attack to achieve a precise outcome.

We extensively evaluate our proposed multi-modal and spatiotemporal attacking approach for its stealthiness, effectiveness, and preciseness, using real-world tracking data from Microsoft HoloLens 2 [1] fed to SelectFusion algorithm. We propose four distinct attacks that increase the speed of a user, decrease her speed, deviate her from a path, and make her follow a longer path to reach the destination. Using the example of a jogger running alongside an avatar trying to maintain a fixed speed on a pre-defined trajectory, we demonstrate that our proposed attacks are stealthy against system checks, effective despite state-of-the-art sensor fusion-based tracking algorithms, and configurable to produce precise outcomes in a changing environment without any future knowledge. We also evaluate our attacks for their robustness in achieving the high-level deviations in speed and trajectory despite the occasional failure of individual frame-level manipulation attacks. Finally, we show that the attacker can dynamically adapt the attacks to launch more sophisticated attacks using techniques presented in the paper.

2 RELATED WORK

In this section, we discuss, and distinguish ourselves from, the related work on MR security, spatiotemporal sensor attacks, sensor fusion algorithms, and securing critical MR applications.

MR Security. Prior research on MR security focuses on vulnerability analysis of application interactions [13, 39, 40], secure authentication and pairing [23, 60, 65], security across- users and devices [54, 89], and privacy in remote collaborations [32, 54]. To the best of our knowledge, no prior work explores the multi-modal attacks on fundamental tracking services that we explore in this paper.

Spatiotemporal Sensor Attacks. There is significant prior work on sensor attacks across multiple dimensions. Data-level temporal and spatial attacks have been demonstrated for the inertial [67, 69] as well as visual sensors [13, 71]. Prior work has also successfully demonstrated the device-level manipulation of timing and location services [8, 9]. Prior work has also demonstrated spatiotemporal attacks that lead to misalignment of content in MR and its inaccurate tracking [60]. The state-of-the-art sensor fusion-based tracking algorithms can overcome these individual data- and device- level attacks, while our proposed concurrent multi-modal attacks are resilient to sensor fusion algorithms and system checks.

Sensor Fusion. The state-of-the-art sensor fusion algorithms leverage deep-learning based tracking on top of visual inertial odometry (VIO), as used by Hololens 2 [1]. Chen et al. [15] present robust deep learning-based multi-modal fusion for VIO by relying on the latent features from different sensor modalities, particularly under degraded environmental conditions such as noise, bias, occlusions, spatiotemporal misalignment, and missing data. In contrast, a malicious spatial and temporal misalignment would distort the fundamental characteristics of latent features, thus presenting new challenges in secure sensor fusion, and motivates further exploration on spatiotemporal attacks which is the focus of our work.

Security of Critical MR Applications. Human-in-the-loop systems like MR are designed not only for fun but also to provide rich experiences and support for users with serious objectives. It can be used to enhance user experience in leisure non-critical applications such as navigating an unfamiliar college campus [47], museum [6, 68], and escape rooms [26, 79] to more critical applications such as manufacturing, logistics, construction [87], warehouse maintenance [50], training, facility management [57], e-learning [5] to support highly critical applications like recreation/fitness [12], driving assistance [51], rehabilitation [21], and surgery [28]. While MR has the potential to improve visual perception in all these applications, it is still heavily dependent on accurate tracking. Tracking is not just to ensure efficacy requirements, but any external malicious attack or internal malfunction can pose significant danger to users. The dangers range from less harmful cases of navigating to a wrong destination to hitting a wall or reaching an unsafe destination that could result in serious physical damage. In future, with more complex and precise applications in medical facilities, where surgeons is using headset in surgery while relying on accurate tracking can to tie medical imagery on top of patient for precise incisions and cuts [28]; or construction worker using headset to share complex structures with remote engineer for guidance [87], then aforementioned tracking inaccuracies may lead to dangerous consequences.

3 PROBLEM FORMULATION

In this section, we motivate the need for launching multi-modal attacks to enable *fast*, *stealthy*, *practical*, and *precise* outcomes. We also outline the problem statement and define relevant key terminologies.

3.1 Problem Context

To iteratively improve and evaluate our attacks, we take a well studied [30, 34, 83] and common real world scenario of jogger (wearing a HMD) running with a virtual partner (avatar) represented through apps like GhostPacer [2], as shown in Figure 1. These apps allow users to set the path and speed of an avatar and race against it outdoors in real-time. The avatar runs on a given route, just like a real partner, and gets closer or farther away from the user depending on how fast the user is running. This ensures that user has an effective run that maintains a desired and safe jogging/running pace. This creates a close feedback loop between the user and the app, user running speed will be recorded through inertial and visual sensors that dictate the tracking and avatar feedback. But ensuring the jogger's safety while presenting appropriate visual information about an avatar is challenging. In our case, the attacker can cause a variety of problems for a jogger, which range from the jogger having a less-effective run/jog (unevenly paced, non-ideal speed [20] and injury [11]) to physically hitting a person or an object. Even though the user can see surroundings but limited field of view (FOV) can compromise the peripheral vision [80], FOV in most HMDs is less than 60°wide – far narrower than our normal FOV of 200°wide [41]. Moreover, an attacker can cause small deviations over a longer time period which would be unnoticeable by the user.

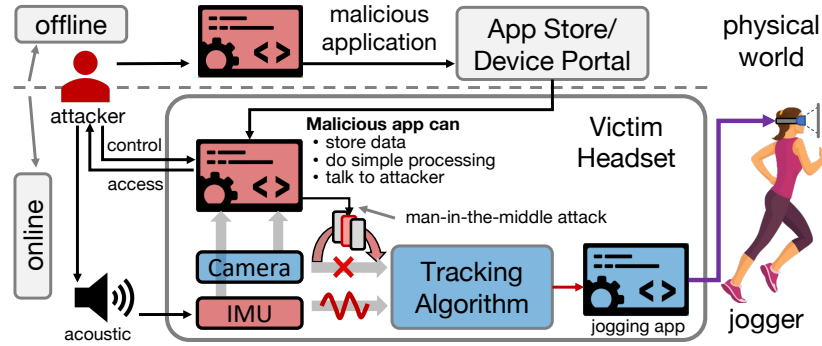


Fig. 3. Threat Model Overview.

3.2 Problem Statement

Before outlining the problem statement, we define the tracking of a user in a MR environment as our *attack surface* with inertial and visual sensor modalities as the *attack vectors* (a carrier chosen by the attacker to exploit the vulnerabilities of *attack surface*). While our attack surface can be generalized for more vectors on the HMD (i.e. audio, infrared), we are considering inertial and visual sensing modalities as our *attack vector* as they provide complimentary temporal and/or spatial features for tracking, that can serve as our *attack knobs*. The *attack knobs* specify the dimensions of an *attack vector* along which the data can be manipulated by the attacker.

Our problem statement is - “how to devise practical attacks that provide precise control of user’s trajectory to an attacker while successfully evading the state-of-the-art sensor fusion-based tracking algorithms and system checks in an MR environment?” To solve this problem, our hypothesis is that *concurrent attacks that use appropriate attack knobs (spatial and temporal) on multiple attack vectors (visual and inertial) enable fast, stealthy, effective, and precise outcomes*. We next outline key challenges in evaluating our hypothesis.

Our proposed attacks (§5.3) rely on simple frame-level manipulations that are not time- or compute-intensive (§5.1) to enable *fast* attacks. We conduct an iterative experimental study to configure the attack to yield *stealthy* (§5.2.2), *effective* (§5.2.3), and *precise* (§5.2.4) outcomes. The outcome of our iterative systematic attack design approach is an algorithm (§16) that, leveraging the insights from controlled attacks done as a part of our iterative approach, determines when and for how long to attack to ensure precise outcomes in a stealthy manner.

4 THREAT MODEL

In this section, we present threat model for the proposed multi-modal attacks. An attacker is attempting to diverge the trajectory of a user wearing MR HMD as shown in Figure 1. There are two kinds of MR HMDs. A user might wear an opaque HMD (Lynx R1 Headset [3]) and accesses physical world through a video stream, where tracking system and entire video stream can be manipulated by attacker. The user might also wear a see-through HMD (Hololens 2 [1]) and directly sees the physical world, where only tracking system can be attacked.

We assume standard attacking capabilities, where the attacker is in close proximity, can visually see the user and the surrounding environment and can use her smartphone to launch context-aware attacks [61, 78], which means that attacker can pick the type of divergence/damage (§5.3) based on user’s current activity. For example, if the user is on the move outdoors wearing HMD, the attacker could still be in close proximity with its smartphone to launch targeted attacks [22]. Also, she does not have physical access to the headset or its firmware but she can obtain a similar device to profile the behavior of sensor data, underlying firmware, sensor fusion algorithms, system verifiers, frame rate of sensors, and architecture. We next describe the main aspects of our threat model and our scenario-specific assumptions in reference to Figure 3.

Malicious App Injection on MR HMD. Our proposed attack methods require reading sensor data as a prerequisite to determining when to attack (details in §5). Internal eavesdropping on Hololens sensor data does

not require any privileges. Motion sensors are zero permission sensors in recent devices [43], accessible by applications and background services without declaring specific permissions. No current operating system allows a user to restrict access of these sensors at a finer granularity [35], allowing the app to capture the data without alerting and/or needing special user permissions [75]. We empirically observe that Hololens does not require user permission for an app to access its motion data, and prior work confirms this [46, 48]. Thus, it opens a hole for a malicious app to collect necessary motion sensor data [43]. The malicious app can reside as a background service app such as fitness tracking app or virtual keyboard app, both of which require continuous access to motion sensor data from inertial and visual streams (our chosen attack vectors), and can run in parallel with other apps.

We assume that an attacker can use a device portal or app store offline to disguise an app using malware-based internal eavesdropping to gain access to the data using malicious app injections shown in prior work [86]. Attacker can use this malicious app to access sensor data in real time by posing as tracking service app [18], it can do computations on sensor data, and form a wireless connection and collude with a nearby attacker [73]. We also consider attackers over the local network that can infiltrate the device portal and access sensor data from the malware[86]. Hololens 2 is a powerful device with ample memory, multiple network channels and computing power (see details in §6.1), so we can assume that a malicious app will not raise alarms by running computations on sensor data, buffering frames, and then eventually sending an attack flag to nearby attackers phone.

Visual attack. The malicious app launches man-in-the-middle attack¹ on visual frames that have shown to be successful on HMDs [60, 64]. Protection against man-in-middle attack can not be guaranteed unless the communication channels [44], data integrity, and user authentication is improved [24, 82]. Given such capabilities, the attacker can easily achieve the visual frame manipulations including duplicating [63], dropping [4], and controlling the change between consecutive frames [72] via man-in-the-middle capabilities [13]. We also assume that she has the ability to buffer a significant amount of visual frames [71]. As our attacks do not rely on forgery and/or occlusions, visual attacks are not flagged by the systems checks.

Inertial attack. While the lack of authentication and digitized nature of visual frames make internal attack on visual stream possible via malicious app, we can not manipulate inertial stream the same way. For inertial attack vector, prior work shows that an attacker can externally manipulate the analog signals on the signal conditioning path, before the data is digitized or processed, by emitting acoustics signals [67, 69]. In §5, we demonstrate how we devise the notion of frames for the inertial data and discuss how manipulations of acoustic signals from prior work [67] can be used to manipulate the inertial frames. Given this notion, the attacker can achieve inertial frame manipulations needed for our attacks that include stretching, compressing, and shifting sensor measurements.

Given the computing power of modern smartphones and embedded devices, it is possible to launch both attacks concurrently. Attacker can use speaker from smartphone to curate inaudible acoustic signals for inertial attacks [84] and can use the same device to control visual attacks launched via malicious app. Furthermore, prior work [13, 63, 67, 69, 71] has already demonstrated that both attacks are individually successful, and hence not in scope of our work. Lastly, all the manipulation when combined across modalities can look like small errors which can be ignored by the tracking algorithm. For example spatial alteration can be perceived as misalignment that can occur due to axis misalignment and the incorrect sensor calibration [42]. The time shift between windows of input images and windows of inertial measurements can be perceived as a relative drift in clocks between independent sensor subsystems [45].

5 DESIGN

This section describes the detailed design of our attacking approach. We first explain the basis of our frame-level attacks and outline our notion of a *frame* for analog signals. We also discuss how to extend the frame-level

¹Man-in-the-middle attack, where the adversary positions herself in between the user and the system to intercept and alter data traveling between them.

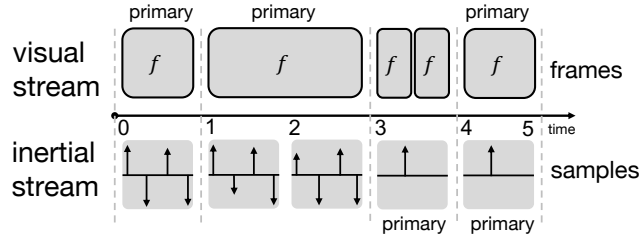


Fig. 4. The notion of a frame changes as the sampling rates of inertial sensors and frames rate of visual sensors change over time. We have to constantly map the inertial samples to visual frames. The primary sensor determines how many visual frames correspond to how many inertial samples.

manipulations, such as adding or dropping a frame, to analog *frames* (§5.1). We then describe our approach to generating *stealthy*, *effective*, and *precise* attacks given the difficulties of avoiding system checks, no explicit coordination across sensing modalities, and unavailability of any future knowledge (§5.2). Afterwards, we propose various practical attacks that leverage the frame-level mechanism and the attack strategy to achieve a specific end goal of the attacker (§5.3). Finally, we integrate all of our components to summarize our attack process (§5.4).

5.1 Physical Basis of Frame Level Attacks

The notion of a *frame* for a visual attack vector is very intuitive. The visual attack vector consists of multiple individual images (also called *frames*) being captured or displayed at a certain rate. The task of manipulating visual frames is also straightforward [4]. You can drop a *frame* from the visual stream, duplicate or add a *frame*, or rotate a *frame*. However, the notion of *frame* does not simply apply to the continuous analog signals that we get from inertial sensors. What do you call a *frame* in an analog signal? How do you drop or add a *frame* to an analog signal? To solve this problem, we devise the notion of an *analog frame* and outline mechanisms for *analog frame* manipulations. While our discussion revolves around an inertial sensor, specifically an accelerometer, the notion and mechanisms apply to other analog sensors such as optical, audio, and pressure sensors.

1. How to view continuous sensor data as a combination of frames? Our first major challenge is to create a shared temporal basis among sensor modalities and their respective data representation to coordinate simultaneous launch of attacks on two different modalities. This is hard because of the additional challenge of dynamically varying data rates of both visual and inertial sensors in practise due to shared resources with system processes and other physical limitations [25]. Our solution is that the sensor with a data point spread over a large time window will act as a primary temporal reference, and the other sensor data will map to that reference. For instance, if there is one visual frame and multiple inertial samples in a given time window, we will choose visual frame as reference and inertial samples will map to it. This mapping is intuitive and simplifies choosing the number of visual frames and inertial samples to add/drop in a particular attack. Figure 4 shows an illustration of our mapping, which is the number of samples or frames within the time window defined by the primary reference sensor. If the frame rates keep changing, the notion of frames will keep changing and our mapping will keep adapting to varying rates of either sensor.

After formalizing the notion of frame for our sensors, our next challenge is to devise frame manipulations for inertial data under the new temporal basis.

2. How to manipulate frames in an attack vector? To answer this, we discuss two concepts and factors: (i) what kind of control we have over the attacked sensor output and (ii) what manipulations we want to perform.

Control over attack output. To do any adversarial manipulations to frames, we need to find a mechanism that causes a mismatch between the true physical property being measured by the sensor (signal) and its digitized version (data). The bottleneck here is to find the opportunity to attack in this signal to data conversion, because once the data frame is reached to the tracking pipeline our attacker can only read the data but can not manipulate it to stay hidden from the system for stealth purposes. One way of masking these attacks is to make attacks

appear as the flaw in the conversion process from physical properties into electrical signals. These attacks can be performed using the interfaces outside of signals intended frequency of operation [25]. We can use interfaces such as electromagnetic radiation exploiting circuits unintentionally acting as receiver antennas [38, 59], as well as optical [53, 59] and acoustic [10, 66] emissions targeting flaws in the conversion process from physical properties into electrical signal.

Usually Microelectromechanical systems (MEMS) based inertial sensors have a sensing mass, connected to springs, that is displaced when the sensor is accelerated. This displacement is translated to a continuous voltage signal. Acoustic waves propagate through the air, and exhibit forces on physical objects in their path [25]. If the acoustic frequency is tuned correctly, it can vibrate the sensors' sensing mass, altering the sensor's output in a predictable way [67, 69]. Any adversary can achieve fine grained control over a sensor's output using this underlined phenomenon, however for our proposed attack surface we use frame level manipulations in order to deliver *fast* attacks. Let's assume that this sensor is subject to a periodic motion or vibration and generates a time-varying signal $S_{acc}(t)$. We generate an acoustic signal, $S_{aco}(t)$, to attack the sensor.

$$S_{acc}(t) = A_{acc} \cdot \sin(f_{acc}t)$$

$$S_{aco}(t) = A_{aco} \cdot \sin(2\pi f_{aco}t + \phi)$$

Here, A_{acc} and f_{acc} are the amplitude and frequency of the generated signal. A_{aco} , f_{aco} , and ϕ are the amplitude, frequency, and the phase of the acoustic attack signal. f_{aco} is set to the resonant frequency of the sensor.

Prior to digitization using Analog-to-Digital Converter (ADC) analog signals are amplified and low-pass filtered (LPF). Typical amplifiers are bounded on both ends; when the input signal exceeds these bounds, signal clipping occurs, and abnormal readings are reported from the sensor. Further, to filter out high frequency signal components LPF prior to an ADC to enforce Nyquist requirement. That states, a minimum sampling rate is required to avoid misinterpreting an analog signal represented in digital form. The final signal sampled by the ADC is the linear combination of the two signals.

$$S(t) = S_{acc}(t) + A_{att} \cdot S_{aco}(t)$$

$$S(t) = A_{acc} \cdot \sin(f_{acc}t) + A_{att} \cdot A_{aco} \cdot \sin(2\pi f_{aco}t + \phi)$$

Here, A_{att} is the attenuation coefficient and is equal to 1 at the resonant frequency. The resonant frequency is the natural frequency of a sensing object where it tends to vibrate at a higher amplitude. To substantially displace the sensing mass, attack frequency (acoustic) must match the mechanical resonant frequency of the sensor. Attacker needs to know the resonant frequency of the signal, f_{aco} , which can be easily determined by profiling a similar signal before the time of the attack. Second, we need to know the magnitude of the signal output, A_{acc} to either increase or decrease it. This is a bit tricky, but can be done in one of two ways. First, by profiling before the attack, attackers could analyze the behavior of an identical device under acoustic effects to find the resonant frequency range and profile suitable attack frequencies of the embedded sensor. Secondly, as noted in 4 most motion sensors have unrestricted access; attacker can get read access to sensor data by posing as an app [86].

Per-frame manipulations For our inertial stream attacks, to drop a single frame, we double the magnitude of the acceleration which can give an impression of users walking longer than actual if applied on an accelerometer. This can be achieved by multiplying the magnitude along each axis by 2.

$$S(t) = A_{acc} \cdot \sin(f_{acc}t) + 1 \cdot 2 \cdot A_{acc} \cdot \sin(2\pi f_{aco}t + 0)$$

Similarly, we can add a single frame by dividing the magnitude along each axis by 2, as it slows down the sensor reading indicating the user is not moving.

$$S(t) = A_{acc} \cdot \sin(f_{acc}t) + 1 \cdot \frac{1}{2} \cdot A_{acc} \cdot \sin(2\pi f_{aco}t + 0)$$

Similarly, we achieve a complete misalignment required for the spatial knob by multiplying the signal with -1, that represents a 180° shift in user orientation.

$$S(t) = A_{acc} \cdot \sin(f_{acc}t) + 1 \cdot A_{acc} \cdot \sin(2\pi f_{aco}t + \pi)$$

Trippel et al. [67] shows that the amplitude modulation can enable a highly precise control over sensor readings to the extent that they spelled out the word WALNUT using sensor output. We use the same underlying principal with additional notion of frame from primary reference sensor such as visual sensor. In our approach we are creating an adaptable mapping to combine any temporal operation such as simultaneous attack on a multimodal sensor between an analog sensor that generates continuous sample (inertial sample) and a digital sensor that generates a discrete frame (visual frame). The control mechanism required for our approach can be launched in practice using the frame to sample combinations between different modalities. Lastly, since adding or dropping a frame in both of our attack vectors is not compute-intensive, both attacks can be launched simultaneously.

3. Matching visual and analog stream manipulations For the visual stream attacker is launching man-in-the-middle attack through the malicious app on HMD and has the capacity to buffer a finite number of packets [71], she can drop frames or duplicate frames in the stream. The camera frame rates for modern MR headsets show significant variations. For example, HoloLens 2's PhotoVideo (PV) camera fluctuates in the range of 5-30 frames per second [1]. Therefore, slow addition or dropping of the frames by the attacker will be interpreted as normal fluctuations. The sampling rate could be different than the frame rate. Sampling rate is usually much higher. If you just drop one inertial sample, it is not the same as dropping an image. Therefore, the frame rate of the visual stream dictates how many samples in the inertial stream we need to drop to achieve an equivalent effect of an image drop. The frame rate of the visual stream can be known a priori, learned through offline profiling, or available through a malicious app in real-time.

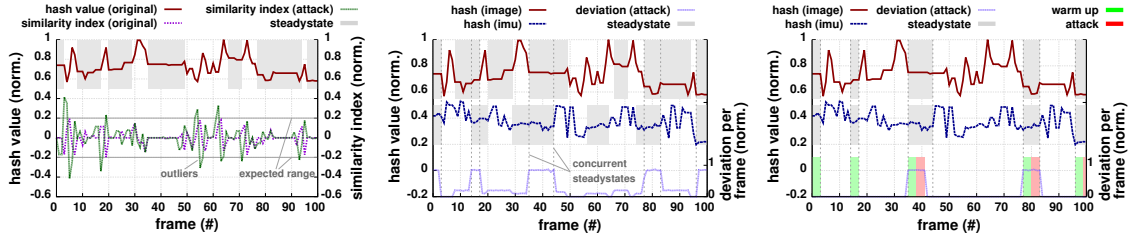
5.2 Iterative Attack Improvement

In this section, we present our approach to designing *stealthy*, *effective*, and *precise* attacks on MR tracking. We start with a naive strategy of launching fast attacks on a per-frame basis and outline the challenges of this strategy in being *stealthy*, *effective*, and *precise*.

5.2.1 Attack Environment. Our attack setup considers an environment where we have two attack vectors, visual and inertial. In our naive attack strategy, we attack each alternative frame by either dropping a frame, adding a frame, or misaligning a frame. These attacks can be achieved by simple frame-level manipulations presented in the last section. We use three different metrics to evaluate an attack's stealthiness, effectiveness, and preciseness. We next outline these metrics and describe their properties in detail.

1. Stealthiness metric. To measure an attack's stealthiness, we use the notion of *similarity index*. Similarity index is a ratio of semantic overlap between adjacent frames f_i and f_{i+1} . A value of 0 means there is no overlap between two frames. A value of 1 means the two frames are the same. Prior work observes that *similarity* between frames is an important metric in determining user pose and semantic similarity for tracking and visual inertial odometry [88]. Therefore, *similarity index* is also used to detect malicious data or erroneous inputs based on the assumption that consecutive frames should be similar. A very low similarity index between consecutive frames means an abnormal change in the scenery, erroneous, and/or malicious data.

There are various underlying matrices that can be used to compare the similarity of different frames. We can use any underlying metrics for the similarity index as long as they quantify the geometrical information for the given attack vector. For visual frames, we use perceptual hashing which examines the contents of an image and constructs a hash value that uniquely identifies an input image [56]. We use hashing as it is robust to minor changes in frames and is used in detecting errors and flagging inappropriate data frames [31, 56]. Given the hash values for frames, we compute the similarity index as the hamming distance between the hashes [37]. For inertial



(a) steadystate enables stealthiness (b) concurrency ensures effectiveness (c) parameterization yields preciseness

Fig. 5. Motivating design decisions: (a) every other frame is dropped as an attack and attacks outside steadystate significantly change similarity index, (b) each modality is attacked whenever it is in steadystate and concurrency of steadystates makes attacks more effective, and (c) parameterizing attacks removes small concurrent steadystates to give more precise control over the deviation.

frames, we use locality-sensitive hashing and scale-insensitive cosine distance, defined as an angle between two vectors, as the similarity index [81].

2. Effectiveness metric. To measure the effectiveness of an attack, we use the *deviation* from the original trajectory as a measure of effectiveness. The attack is successful if it results in a *significant* deviation from the trajectory. The definition of *significant* is context dependent and may not be the same for the same configuration of an attack under different scenarios. However, it should have a simple property of increasing monotonically as the attack is launched more frequently or for a long duration.

2. Preciseness metric. To evaluate preciseness of an attack, we use the ability to achieve a desired and consistent deviation whenever an attack is launched. We define it as the ability to launch an attack of a given length each time and get the same deviation each time.

5.2.2 Stealthy Attacks. As described in the attack setup, our naive strategy launches attacks on every other frame. In this case, we drop every alternative frame. Figure 5a shows the hash value and similarity index of visual attack vector. We observe that whenever the scene is changing significantly, the hash values are also changing significantly. However, despite the scene changes, the similarity index between consecutive frames always lie within a tight bound as no erroneous or malicious data is observed. However, if the attack is launched during the scene transition periods, the similarity index can change significantly because dropping a frame can cause significant difference between the consecutive frames in the attacked stream. The system checks can catch this as an abnormality and inform the user about a potential attack or a glitch or close the application. On the other hand, whenever the scene is not changing significantly, the alternative frame dropping attack does not produce similarity index outliers. This results suggests that we need to consider the state of attack vectors to decide on the times when to attack to be stealthy. The same metric that can potentially be used to catch the attack can also inform when the attack is to be launched.

Key takeaway. Attacks on attack vectors when the scene is changing significantly can be caught by the system checks and fast attacks need to be launched during the steady states of the attack vector to ensure stealthiness.

5.2.3 Effective Attacks. Previous section outlined “how” to attack a single vector and discussed “when” should these attacks be launched on it. However, sensor fusion and tracking algorithms leverage multiple attack vectors and their steadystates may not always align. There could be scenarios when only one of the attack vectors is in steadystate while the other is not. We next investigate the effectiveness of an attack when it is launched on each attack vector individually whenever they reach a steadystate. Figure 5bb shows two attack vectors: visual and inertial. As opposed to Figure 5ba, the attacks are launched only during the steadystate windows. We also show the deviation achieved whenever an attack is launched. There are two observations to be made from this figure about the effectiveness of attacks. First, whenever an attack is being launched on a single attack vector, the deviation is minimal despite the severity of the attack. However, when the attack is launched on both streams at the same time, the deviation is much larger than the sum of deviation for individual attacks. This means that the

sensor fusion based tracking algorithm is able to mitigate the individual attacks by relying on the other attack vector. However, when both streams are attacked, it ends up amplifying the result of the attack. Second, launching attacks when both attack vectors are in steadystate provides a frame of reference for attacks on individual attack vectors to align and start at the same time. It also provides the attacker an ability to launch attacks only in desired certain contexts, e.g. when the scene is not changing.

Key takeaway. *Sensor fusion and tracking algorithms are able to mitigate attacks on individual streams but concurrent attack on both streams yields deviation greater than the sum of deviations for individual attacks.*

5.2.4 Precise Attacks. In Figure 5b, we observe that there are multiple windows of variable lengths where the steadystates of the different attack vectors align. In our attack setup for that experiment, we assumed that we had a complete future knowledge and when the steady states windows started and ended. However, in the real world, there is no way of knowing when a steadystate window starts and determination of steadystate can be done only on the past data. As a result, the attacker needs to solve two problems: first, to estimate the start of the steadystate and second, to configure when to end the attack within the steadystate.

We solve these problems using a setup based on our threat model. We assume that the attacker keeps a record of either the hash values of frames or their similarity indices. Given this historical data, the attacker defines a *normal* range for the similarity index using two values: similarity index (SI) upperbound and lower bound. If the value of similarity index for arriving frames remains within this range for a certain number of frames, which we call a *warm up* length, the attacker concludes that the attack vector is in steady state. Furthermore, once steadystate has started, it only attacks for a certain number of frames, which is defined by *attack length*. Figure 5c illustrates this scenario where the attacker waits for the warm up period (green windows) and once the steadystate is established, it attacks the next pre-defined and limited number of frames. This approach solves two problems. First, it removes the short windows of concurrent steadystates and second, it terminates the attack during large steadystate windows to achieve a specific deviation level. In addition, both parameters provide a tradeoff between the stealthiness, effectiveness, and preciseness of the attacks. The attacker can set the warm up period to be 0 and attack more often, the attack will be effective in achieving deviation but will neither be precise nor stealthy. On the other hand, a large warm up period will leave only a few steadystate windows for attack making it precise and stealthy but less effective. The opposite trade-offs are true for the attack length: longer attacks are effective but neither stealthy nor precise, shorter attacks are less effective but stealthy and precise.

We next formalize this approach by designing an algorithm to determine when to attack, which we term as Right Frame Selection (RFS) algorithm. The workflow of the algorithm is shown in Figure 6 and pseudo code is shown in Algorithm 1. We next define the key terminologies used in the algorithm.

- **History Length (history_length):** The number of frames an attacker stores to decide on steadystate bounds.
- **SI upperbound (SI_upperbound):** The $X\%$ ile value for the stored history. X is determined by offline profiling.
- **SI lowerbound (SI_lowerbound):** The $(1-X)\%$ ile value for the stored history. X is based on offline profiling.
- **Warmup Length (warmup_length):** The number of frames that need to be within bounds to declare steadystate.
- **Attack Length (warmup_length):** The number of frames that are attacked within a steadystate.

Right Frame Selection (RFS) Algorithm. Algorithm 1 shows an automated mechanism for picking the frames to attack while staying stealthy. It takes a stream of *frames* and other attack parameters (configured by the attacker) as an input and outputs the *attack_flag*. If the *attack_flag* is 0, it is not safe to initiate or continue an attack. If the *attack_flag* is 1, the attacker can initiate or continue the attack. To ensure concurrency, we only launch an attack when the *attack_flag* is 1 for all the attack vectors.

The inner mechanism of the RFS algorithm is purposefully simple. It buffers the incoming frames in a First In First Out (FIFO) buffer of size *history_length*. Once the history buffer is full, the algorithm computes the similarity index for all the frames in the buffer. If the similarity index lies in the specified range, a counter for the steady-state is updated. A steady-state is reached if the scene is consistent and the similarity index remains in

Algorithm 1: Right Frame at the Right Sensor algorithm: Configuring the Start and the End of an Attack.

Input: frames, history_length, warmup_length, attack_length, SI_upperbound, SI_lowerbound,
Output: attack flag

```

1 attack flag  $\leftarrow$  0
2 steadystate_counter  $\leftarrow$  0
3 attack_counter  $\leftarrow$  0
4 history  $\leftarrow$  FIFO buffer: [0: history]
5 while frames arrive do
6   history.append( frame)
7   if (history is full) then
8     get similarity index (SI) of adjacent frames
9     if (similarity index is within [SI_lowerbound, SI_upperbound]) then
10      | increase steadystate_counter
11    else
12      | steadystate_counter  $\leftarrow$  0; attack flag  $\leftarrow$  0
13    if (steadystate_counter  $\geq$  warmup_length) and (attack_counter  $\leq$  attack_length) then
14      | increase attack_counter; attack flag  $\leftarrow$  1
15  else
16    | attack flag  $\leftarrow$  0

```

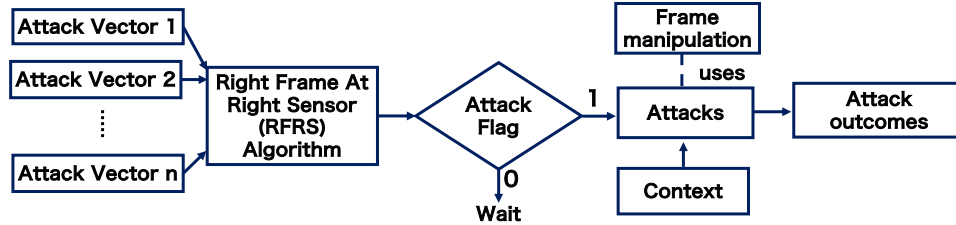


Fig. 6. Our approach to launching concurrent fast, stealthy, and precise spatiotemporal attacks.

the specified range for a period of warmup_length. Once the steady-state is achieved, the RFS algorithm will signal the attacker to initiate the attack. The attack will continue until the similarity index value goes outside the specified range or the number of frames attacked reach the attack_length. In this case, all the counters are reset and the algorithm starts searching for the next steady-state.

Key takeaway. Our configurable attacking approach provides the attacker a controllable tradeoff between the effectiveness and the preciseness of the attack. The attacker needs to configure the right balance between the two.

5.3 Attack Generation/Method

Given the basic frame-level manipulations and our attack strategy, we next present various attacks that achieve a concrete and quantifiable goal for the attacker. We designed four attacks to demonstrate the validity of our attack surface. The goal of the attacker is to diverge the user's "reality" by simultaneously manipulating at least two attack vectors. The choice of manipulations depends on the attack requirements. We next describe the proposed attacks (summarized in Table 1).

Slowdown Attack. In this attack, an attacker temporally *stretches* the attack vector by adding more frames between two consecutive frames, as demonstrated in prior work [63]. For the visual vector, the attacker will duplicate the images sent to the sensor fusion algorithm's visual processing pipeline. For an inertial vector,

Attack	Attack Mechanism	Attack Vector	Attack Knob	End Effect
Slowdown	Stretch	Camera	Temporal	Applications perceives user slowdown and prompts user to walk/jog faster.
		IMU	Temporal	
Speedup	Compress	Camera	Temporal	Applications perceives user speedup and prompts user to walk/jog slower.
		IMU	Temporal	
Zero Displacement	Stretch, Compress, Shuffle, Histogram Shuffle	Camera	Temporal, Spatial	User reaches the destination through a different path.
		IMU	Temporal	
Path Deviation	Histogram Shift, Misalign	Camera	Spatial	User is unable to reach the destination.
		IMU	Spatial	

Table 1. Summary of proposed multi-modal spatiotemporal attacks in MR tracking and navigation.

the attacker reduces the amplitude of the inertial sensing signal to create the “add frame” effect. Reducing the amplitude to 0 is equivalent to adding infinite duplicate images for the visual stream. To do so at time i , she can decide to hold the last frame that arrived at time $i - 1$. Alternatively, as she has a buffer capacity, she can replay a sequence of frames from the history. In our design, we use the holding approach as it is simpler and faster to implement in practice. Note that the same attacking approach is concurrently launched against both the inertial and visual streams. Figure 7(a) shows the non-attacked sequence of frames. While Figure 7(b) shows the sequence of frames after the attacker has induced two copies of malicious frame f'_1 between frames f_1 and f_2 . The end effect is that the pacer app perceives the user has slowed down. For instance, the attacker can use a slowdown effect on a user trying to jog at a fixed pace. This slowdown effect will deceive the pacer app to estimate user jogging slowly, and respond by making partner avatar faster to encourage jogger to run faster leading to harmful consequences [36].

Speedup Attack. The goal of this attack is opposite of the slowdown attack. In this approach the attacker manipulates frames temporally. She *compresses* the attack vectors by dropping frames, as demonstrated in prior work [33]. For the visual stream, the attacker will not forward some of the packets, which is akin to a camera operating at a lower frame rate. However, the scene in the images forwarded would be changing fast. For the inertial stream, the attacker can increase the magnitude of the measurements to give an impression of dropped frames. Increasing the magnitude to the maximum value is functionally equivalent to generating images at the highest frame rate for the visual stream. As shown in Figure 7(c), as frames drop, the sequence of frames seen by the tracking algorithm is changed. For example, if the attacker drops f_2 and f_3 , f_4 will become new f'_2 and so on.

This packet drop will give the application an impression of the user speeding up. In our example scenario, the speedup illusion caused by the attack may result in the pacer app asking the user to jog at a slower pace by making partner avatar slower, leading to an ineffective jogging session [11].

Zero Displacement Attack. This attack takes the user from the *source* to the *destination*, but through a different path, resulting in a zero displacement for the overall trajectory. This attack uses a combination of *stretch*, *compress*, *shuffle*, and *histogram shift* mechanisms to ensure that the distance traveled by the user is also the same as the original trajectory. Shuffle approach is used by attacker to temporally switch between adding (stretch) and dropping (compress) frames, as shown in Figure 7(d). For example, an attacker can drop f_2 at its original time and buffer it to use it in place of f_6 . It may also add some forgery in the frames to corroborate with the current tracking scenario. Attacker may manipulate an image frame such that the difference in histogram between adjacent frames does not exceed a threshold, whose value is determined empirically. Histogram of an image shows the density distribution of pixel intensities and provides information about the objects in a given image. One method to launch this attack is where the attacker takes a difference between the current frame i and the previous frame $i - 1$. She then uses edge detection to identify the segments of the image that are different from the previous frame. Finally, she applies this “difference image” to the new image such that the histogram difference does not exceed the aforementioned threshold value. The goal of finding and respecting the right threshold is to avoid making drastic changes and remain stealthy. This allows the attacker to manipulate the spatial map of the surroundings leading to inaccurate positioning and tracking, as shown in Figure 7(f). Unlike the previous two attacks that just

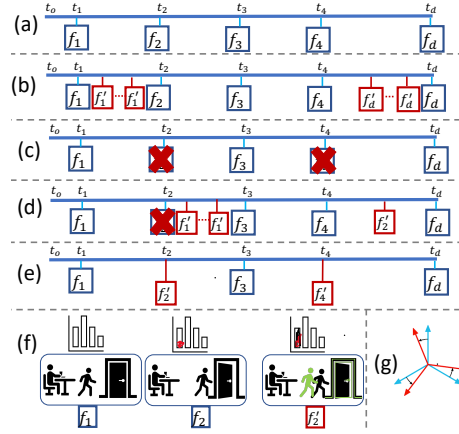


Fig. 7. Various frame manipulation techniques. (a) original, (b) stretch, (c) compress, (d) shuffle, (e) visual data misalign, (f) histogram shift, and (g) inertial data misalign.

used temporal knobs, this attack also exercises the spatial knob using histogram shift to mask the differences in the frames when they are shuffled. This attack is useful when the attacker wants the user to go through a specific point outside the original route defined in the pacer app or she wants the jogger to run at an uneven pace.

Path Deviation Attack. In this attack, the attacker *misaligns* the spatial locality of the visual and inertial frames. The attacker employs *misalign* mechanism to attack inertial vector and employs *misalign* and *histogram shift* mechanism to attack visual vector. Tracking algorithm looks at the f_i to f_{i+m} frames to determine the position and orientation of the user, where m is the window size of history for the tracking algorithm [58]. For the visual frame, it can be achieved by adding padding to the image on the desired side and misaligning the location of objects in the environment and then employing *histogram shift* to mask it. For the inertial frame, it can be achieved by inverting the inertial sensor measurement. Figure 7(e) depicts a visual misalignment while Figure 7(g) shows the misalignment for the inertial sensor. The tracking application is fed the wrong pose information about the user, which results in an incorrect estimation by the pacer app. As a result, the partner avatar changes its path trying to reach the destination, hence deviating the user from the original path and causing the user to lose the destination. It is easy to envision how this can be very dangerous for jogger especially in any unfamiliar and/or unsafe location.

5.4 Putting All Things Together

We now have described all the ingredients for launching *stealthy*, *effective*, and *precise* attacks. The ultimate goal of the attacker is to deviate the user of the jogging application either from its trajectory or its desired speed. To achieve this goal, we present an end-to-end attack pipeline in Figure 8.

Our attack pipeline has two stages. In the offline stage, the attacker needs to install a malicious application on the victim's device that has read-only access to the attack vectors (camera and inertial) sensor on the victim's device. In the same offline stage, the attacker needs to determine the typical frames rates for both the inertial and visual attack vectors to configure the notion of frames. It can also employ frame manipulations on the available device to profile a relationship between the attack parameters of RFS algorithm and the deviations they achieve. Given this, the attacker can generate a wide range of attack strategies and collect various configuration parameters that can be used during the online stage of the attack.

In the online stage, the attacker sets a desired goal for the attack such as *slowing down the user*. It then uses offline profiled information to determine how many frames need to be dropped to achieve the desired level of deviation depending on the context of the scene. The attacker meanwhile gets the data from the malicious

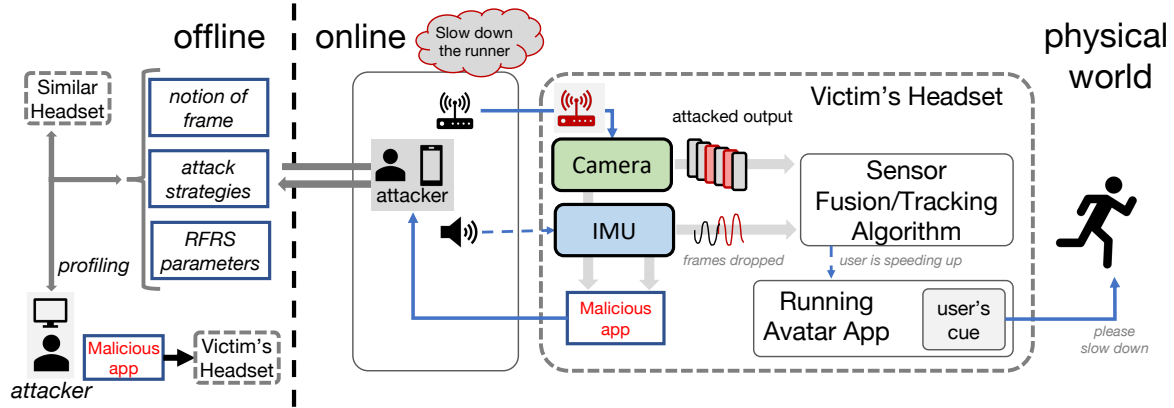


Fig. 8. Multi-modal concurrent attack strategy overview consisting of offline profiling and online attack stage.

application to decide on when to attack. The attacker does not necessarily get all the data from the device, as it can be network-intensive and may cause delays. The attacker can store the data inside the malicious application and only send high level statistics such as the upper bounds and lower bounds of the similarity index for the RFS algorithm. As the attack is launched on the attack vectors, the tracking algorithm is fooled to believe that the runner has started running faster. As the goal of the application is to maintain a fixed speed or run alongside an avatar, it provides feedback to the user to slow down and in turn achieves the goal of the attacker.

While we have discussed attacks that allow the attacker to speed up, slow down, or deviate a user, they also provide the foundation for more sophisticated attacks on safety critical applications. For example, an attacker can launch a series of above attacks in different order to achieve the desired malicious tracking.

6 EVALUATION

In this section, we evaluate the stealthiness and preciseness of our proposed concurrent multi-modal attacks.

6.1 Experimental Setup

Data. To collect real-world data, authors of the paper mimicked jogger with a partner avatar jogging around a variety of environments (indoor, outdoor) and scene setups (trails, suburbs, downtown) under two locomotion scenarios (walk, jog). Tracking data and the ground truth is collected using Hololens 2 operating in research mode [1, 70] with Qualcomm Snapdragon 850 compute platform holographic processing unit, 4GB LPDDR4x system DRAM, 64GB universal flash storage 2.1, a depth camera, an RGB camera, four grayscale visible light tracking (VLC) cameras, and an inertial measurement unit consisting of an accelerometer, gyroscope, and magnetometer. VLC generated data at a frame rate of 5-30 frames per second (fps) and raw inertial sensor data was collected at 12-20 fps, both with timestamps [70]. We matched the timestamps of closest inertial data to the VLC data frame for sensor fusion. Overall, we collected 154032 samples (2.5 hours with mean 17 fps) of data.

Tracking. Tracking algorithms are the key component of any positioning and tracking application. Our attacks manipulate sensor data, and that malicious data is fed to the tracking algorithm to mislead its estimate of the user's actions. In our evaluation, we use the state-of-the-art sensor fusion-based tracking algorithm Select-Fusion [15] that is highly effective against attacks on single sensors, as shown in Figure 2. Our tracking module takes raw images and inertial frames to generate corresponding pose transformations. We train our model on our 154032 data samples with 75% - 25% train and test split. We base our implementation architecture on Select-Fusion and train in the stochastic fusion mode. We use PyTorch for implementation and trained on a NVIDIA GeForce RTX 2070 GPU with batch size of 8 using the Adam optimizer, with a learning rate $lr = 1e^{-4}$. Training and testing mean squared errors was 0.0095 and 0.017 for Select-Fusion, respectively.

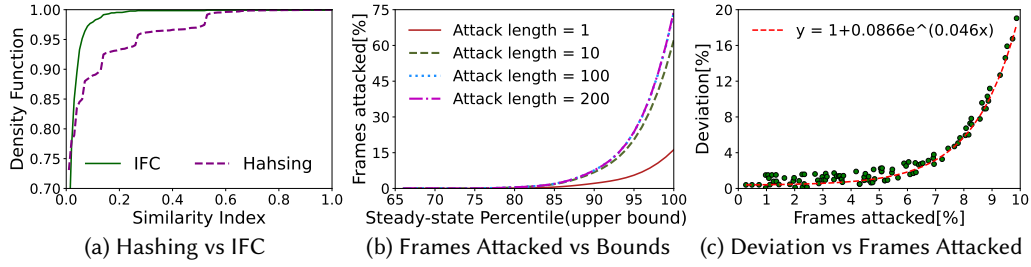


Fig. 9. Configurability and preciseness of our Attack Surface.

Metrics. To evaluate how the overall system perceives the effect of our attacks over time, we use the Cumulative Density Function (CDF) of the similarity index over the attack sequence and compare it with the original sequence. CDF is commonly used to detect the attacks on the system [52] because typical frame distributions and relationship with subsequent frames do not change drastically in a short period of time. Our assertion is that if the overall frame distribution has not changed, the system is not perceived to be under attack by any system checks.

To quantify the similarity between different distributions, we compute Kolmogorov–Smirnov test [49]. For stealthiness, the attacker needs to keep this as small as possible as it represents how close are the distributions for the attacked and original sequence. We quantify the difference between arithmetic means of both sequences to find the variation among similarity indices. Higher the mean difference, more the difference between sequences.

6.2 Configurability and Preciseness of Attacks

Our approach offers an attacker multiple configurable parameters to launch precise attacks. These parameters determine what percentage of frames are attacked and how many frames are attacked in one attack sequence. To configure these parameters, she needs to know the relationship between the parameters and the ultimate deviation achieved in the user’s trajectory. To quantify this relationship, we perform a set of experiments that vary the parameters and record their effect on trajectory.

Sensitivity Analysis of Similarity Index Metrics. Our attack approach relies on similarity between frames is an important metric in determining user pose and semantic similarity. We analyzed additional similarity metrics for their ability to capture the nuances in our scenario. We compared our chosen metric hashing with Inter Frame Correlation (IFC). IFC measures the correlation among consecutive frames. Figure 9a shows the CDF of hashing and IFC for a single sequence in our use case. IFC is unable to detect the different nuances of frames similarity index, but hashing paints a nuanced picture of our use case.

Effectiveness of Attack Strategy/Impact of Steady-State Window. Figure 9b quantifies attackers’ strategy to figure out how long the steady-state should persist before attack starts to ensure an effective attack. This can be visualized in the relationship between steady-state bounds and the steady-state length parameters. To reiterate, steady-state bounds define the acceptable range for the similarity index values. Steady-state length controls how long the similarity index needs to be in range for a sequence to be labeled in steady-state. We can see that if the bounds are set too low, there are not enough sequences of frames that can be attacked. For example, almost no frames can be attacked if the lower-bound and upper-bound are set to 20% and 80%, respectively. The number increases significantly at 90% upper-bound value and encompasses almost all the frames if the bound is set to 100%. Here the different curves show the effect of the steady-state length on the frames attacked. The total frames attacked increases as the attack length increases from 1 to 10, but there is only a marginal increase beyond that.

Stealthiness versus Deviation Trade-off. Figure 9b shows how many frames are attacked, but does not inform the attacker on how much deviation will be achieved as a result. To analyze that, we varied the number of frames attacked (by carefully configuring the attack parameters) and measured the deviation achieved under a wide-range user movement scenario. Figure 9c shows the actual data points and the fitted exponential curve to the data. We see that the deviation increases exponentially as the number of frames attacked increases. However,

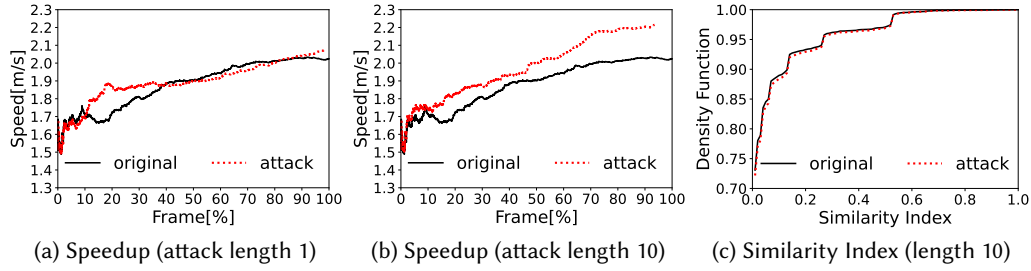


Fig. 10. *Speedup Attack. The magnitude of Speedup depends on attack length (small at 1 (a) vs large at 10 (b)). (c) shows that the distribution of hashing function that does not change before and after the attack demonstrates stealthiness.*

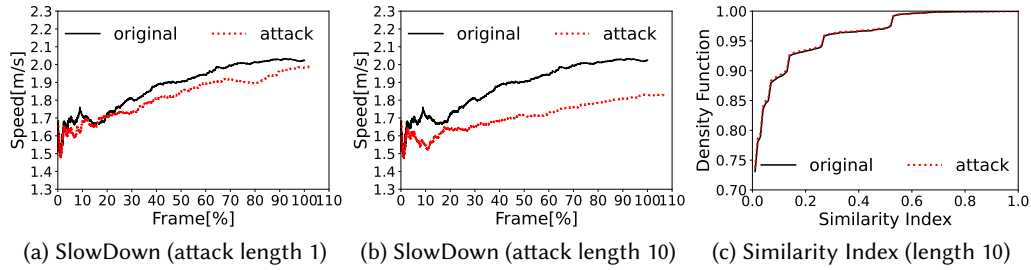


Fig. 11. *SlowDown Attack. The magnitude of SlowDown depends on attack length (small at 1 (a) vs large at 10 (b)). (c) shows that the distribution of hashing function which does not change before and after the attack, demonstrating stealthiness.*

the exponential relationships dictates that the attacker would need to be cautious in devising the attacks as marginal change in deviation can be very large. Furthermore, as our threat model assumes that the attacker has access to a similar headset, she can determine this relationship offline. The relationship stays the same if the environment stays the same. However, the relationship may change if the environment changes and the attacker would either determine the new relationship empirically or get this equation from a third-party attacker.

Our evaluations point to a trade-off between risk and reward for the attacker. For a large deviation, she has to attack a large %age of frames, which may expose the attack. A small attack is stealthy but may not achieve the desired outcome. This is further complicated by non-linear increase in deviation for linear increase in parameters. Therefore, the attacker should remain in a range that offers a more steady relationship between the attack parameters and outcomes. In our use case, this range lies between 85%-95% steady-state bound and the attack length of 1, where she attacks only 10% of the frames and achieves a deviation in the range of 2-20%.

6.3 Efficacy and Stealthiness of Attacks

In this section, we evaluate the efficacy, preciseness of control, and stealthiness of our proposed attacks.

Speedup Attack. In a *speedup* attack, the attacker wants to give the tracking application (partner avatar app) an impression that the user is walking at a faster speed than her actual speed. As the application tries to maintain a constant pace for the user, it will instruct her to reduce her speed by making the partner avatar slower, leading to an eventual slow down of the user. Figure 10 shows the effect of *speedup* attack on cumulative average speedup and similarity index distribution. To reiterate, an *speedup* attack is achieved by dropping frames causing the scene to change more rapidly. The tracking algorithm does not perceive this as a gap in time, but instead as the user moving faster. Figure 10a shows that the end gain in speed is very small after the attack as only a small number of frames were dropped. As the length of the attack is increased to 10, the increase in the perceived speed is significant as a larger number of frames are dropped, as shown in Figure 10b. It is worth noting that our attacks are successful at both the small as well as a slightly larger attack lengths. This is because the tracking algorithm is unable to correct for the deviations at both scales due to the concurrent and multi-modal nature of the attacks.

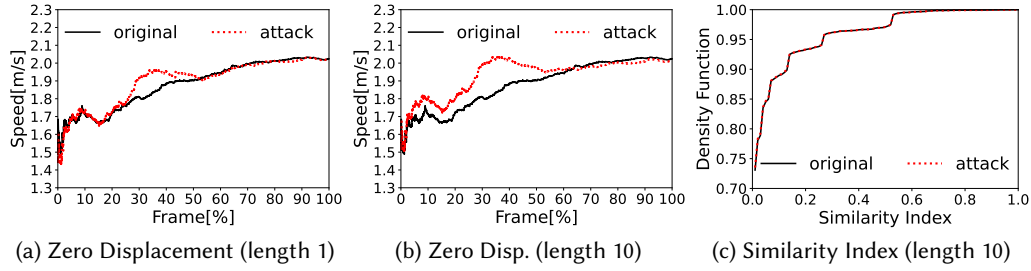


Fig. 12. Zero Displacement Attack. The magnitude of deviation depends on attack length (small at 1 (a) vs large at 10 (b)). (c) shows that the distribution of hashing function which does not change before and after the attack, demonstrating stealthiness.

Figure 10c demonstrates the stealthiness of *speedup* attack. While the attacker is able to increase the speed by around 0.3m/s , it does not significantly change the overall distribution of similarity index. A sanity check mechanism that uses change in similarity index across frames would not be able to detect the attack. As mentioned above, the final effect of the *speedup* attack depends on the application characteristics. In the alternative scenario, the user trajectory will change if the application is trying to guide the user on a specific path. This will happen as the sequence of frames seen by the tracking algorithm, which relies on a time-series of data, changes post-attack. **Slowdown Attack.** In this attack, we consider the same example as the previous attack. The expected result for a *slowdown* attack is that the application will perceive the user as walking slowly and generate a feedback for her to walk faster through the avatar, leading to an actual *speedup* effect.

Figure 11 shows the effect of *slowdown* attack on the cumulative speed of the user and similarity index distribution. To reiterate, a *slowdown* attack is achieved by holding the frames causing the scene to change slowly, giving the application a perception of the user moving slower. The observed results are very similar to the previous attacks. At small attack lengths, slowdown is small and only a few attacks are added (Figure 11a). At larger attack lengths, slowdown is large and large number of frames are added (Figure 11b).

Figure 11c demonstrates the stealthiness of the launched attack. Similar to the *speedup* attack, the attacker achieves a speed slowdown of around 0.3m/s , without significantly changing the distribution of the similarity index. While an instruction to walk faster defeats the purpose and deviates users from the trajectory, it can also have adverse physical consequences for people with underlying health conditions and bystanders in the situation. **Zero Displacement Attack.** In a *zero displacement* attack, the attacker wants to give the application an impression that the user is periodically changing its speed, walking faster at times while slowing down at others. Since this attack is launched using a combination of strategies used in *speedup* and *slowdown* attacks, the end effect depends on when each of the attacks is launched and if one type of the attack dominates the other. If both attacks are launched alternatively for the same duration, the net effect would be zero. The user would have just followed a slightly different path to the original trajectory.

Figure 12 presents the results for zero displacement attack. Figure 12a and 12b show that speed increases slowly at the start, followed by a big divergence, and finally reduces towards the end. This constant change in speed can diminish the benefits of a jogging partner avatar app. Moreover, it can result in an ineffective fitness session and/or injury [36]. In practical scenarios, where the attacker would not know the speed of the user in advance, so constant shuffling of frames can get the attacker to the final destination at the same time. However if this attack is handled only with a temporal knob the attacker might have diverted from the original destination even with shuffling of the frames. The attacker uses spatial knobs to bring the user back to the final destination. Similar to the previous two attacks, Figure 12c shows that there was no statistically significant difference between the similarity index of the original and attacked sequences.

Path Deviation Attack. Our final attack, the *path deviation* attack, demonstrates the effect of manipulation via spatial knobs on both the inertial and visual attack vectors. Here, the attacker wants to change the direction to

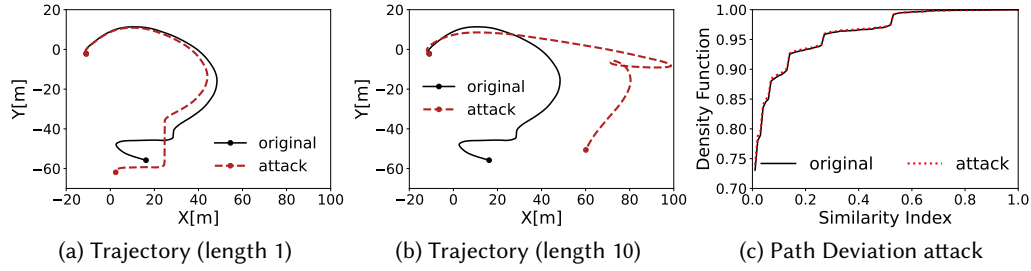


Fig. 13. *Path Deviation Attack. The trajectory is shown with attack length of 1 (a) vs attack length 10 (b). (c) shows that the distribution of hashing function as a similarity index does not change before and after the attack, demonstrating stealthiness.*

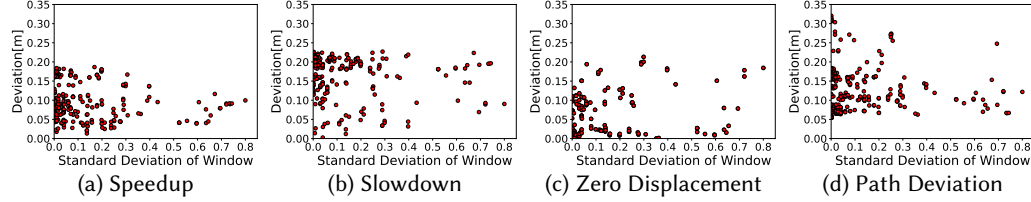


Fig. 14. *Effect of steadiness of a given window on the deviation achieved in that particular window. Low variation in a given window results in a more controllable and effective attack.*

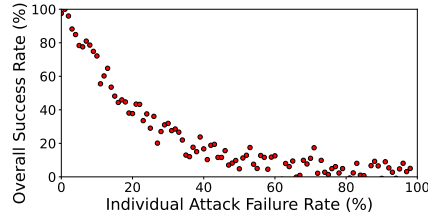
deviate her from the final destination. To reiterate, an *Path Deviation* attack is achieved by changing the alignment of the frames, leading to a perception of the user moving in a different direction and ultimately reaching a different destination than initially intended. Figure 13 shows the effect of *path deviation* attack on cumulative user speed and similarity index distribution. Figure 13a shows that the trajectory after the attack shifts by a small margin at small attack length. As expected, the change is significant for the larger attack length, as shown in Figure 13b. This is the long lasting effect of smaller changes throughout, the attacker did not change much but kept changing for longer duration and slowly the effect accumulated at the end. Overall the attacker achieved a 32 meters change in position and 1.5 radians change in the orientation. Finally, Figure 13c shows no significant difference between the similarity index of original and attacked sequences.

6.4 Effect of Steadiness on Attacks

As mentioned before, our proposed attacks are launched during the steady-state, which is defined as having numerous consecutive frames that lie within a certain bound. However, not all steady-states are the same and may have significant variations in the frames within the steady-state window. For example, if a scene is changing, frames would be similar and vice versa. We quantify this variation using standard deviation calculated over a given steady-state window and analyze the effect of this value against the deviation for that window.

Figure 14 shows that, across all proposed attacks, most of our attack sequences are launched in a window with lowest standard deviation between the similarity index. Some of the attacks are launched in highly variable windows as well. This can happen as a highly dynamic scene slowly transitions into a quiet scene. The steady-state can be achieved despite the high variations in the similarity indices, leading to an attack being launched.

Figure 14 also illustrates the impact of standard deviation in similarity indices on the deviation achieved. In most cases the low standard deviation, aka extremely stable steady-state, results in high deviation. This facilitates stealthiness and yields divergence intended by the attacker. This demonstrates a significant potential of our proposed attack surface and algorithm. MR is becoming common in critical applications like surgery [28] and rehabilitation [27]. In such applications, micro movements are tracked and the physical scene does not significantly change but movements are critical. Our attack surface works best in steady state and the attack will be effective even in scenarios where the micro movements are essential, and attacking those can cause serious consequences. Even if the scene is not changing the deviation is still attainable in those scenarios.

Fig. 15. *Robustness against attack failures.*

6.5 Robustness

We next demonstrate the robustness of our attacks against frame-level attack failure and RFS algorithm errors.

Robustness to Attack Failure. Our attacks are based on sensor data manipulation and the attacker needs to carry out these attacks in a short amount of time (on average 33.3 ms, more on this in §6.8). Use of physical medium and multiple attacks concurrently can sometimes create unfavorable situations for an attacker. For example an attacker launches attack on both modalities concurrently when RFS algorithm deems that frame attack is stealthy, but sometimes individual attack on one or both modality can fail due to physical environment or any technical issue. In Figure 15 we demonstrate the effect of these individual attack failures on a given frame in a series of attacks for a particular scenario. We measure success rate here as total deviation caused as a result of attack when %age of individual attack failed divided by the total deviation caused by an attack when no individual attack was failed. We carried out 100 attacks where we deliberately failed some %age of attacks. We can see that when 20% of individual attacks fail with our approach, the attack can still attain $\approx 40\%$ of intended divergence. For example if attacker is looking to slow down a user by 20% of its initial speed from 1.5m/s to 1.2m/s (as demonstrated in §6.3), even with 20% failed individual attacks she will be able to slow down user to ≈ 1.4 m/s. While the divergence exponentially drops with the increased individual failure rate, in practice even 20% of these simple individual attack failures is an overestimate [13, 67]. We will investigate this further as part of our future work.

Robustness of RFS Algorithm We next demonstrate that if the scheme to pick the frames to attack is steadiness-agnostic, it will be less effective, less configurable, and less stealthy. To evaluate this, we use three different steadiness-agnostic periodic frame-picking schemes. These schemes include (1) *FixedLength*, where the attacker launches the attack periodically after every N frames for a fixed attack length, e.g. 10 frames, (2) *EqualSum*, this schemes launches the attacks periodically but makes sure that the number of attacked frames is same as ours RFS algorithm. (3) *XPercent*: in this scheme X% frames of the sequence are attacked.

Figure 16 demonstrates the use of periodic frame-picking schemes to launch a *slowdown* attack. Figure 16a shows that, using *FixedLength* scheme, the attacker is not able to launch a consistent *slowdown* attack, as was the case for RFS algorithm (Figure 11b). Furthermore, the total attacked frames are not equal to RFS algorithm with attack length of 10. For a fair comparison, we used the *EqualSum* scheme where the total number of attacked frames is as RFS. Figure 16b shows that the deviation increases, but attacker is not able to launch a precise attack.

The final configuration of attack is shown in Figure 16c. While this attack seems highly effective, the graph does not paint a complete picture. Table 2 presents a comparison of attacks we have launched based on various metrics. We can see that the *ks* test value and the mean difference for the *XPercent* attack is the highest, which means that this attack is the least stealthy attack in our set. This will result in the attack being detected by the system, as it has changed the distribution of the similarity index for the sequence significantly after the attack.

6.6 Dynamic Adaptation of Attacks

In prior evaluations, we selected a given attack, such as *slowdown*, and persisted with the attack for the entire duration. However, under a fast-changing environment or user behavior, the attacker may want to adapt the attack based on the scene or user's action. This necessitates that the attacker should be able to switch between the attacks at runtime, without sacrificing stealthiness, control, and preciseness of the attacks. Since the underlying

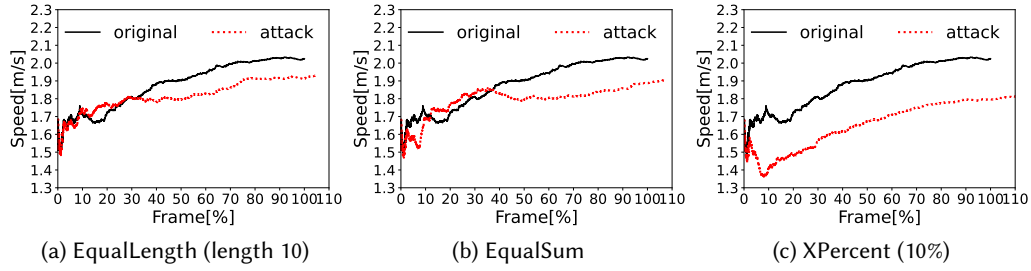


Fig. 16. Difference between the speed on attacked sequence and original sequence.

Attack	Attack percentage	ks test statistic	Mean difference
SlowDown	7.2	0.06	0.14
Speedup	7.2	0.07	0.11
Zero Displacement	7.2	0.05	0.03
Path Deviation	7.2	0.07	0.15
FixedLength	4.5	0.07	0.23
EqualSum	7.2	0.08	0.35
XPercent	10	0.13	0.49

Table 2. Comparison of different frame picking schemes

attack mechanisms make changes on per-frame-basis, and do not assume anything about the future, different attack mechanisms can be selected for each frame. Furthermore, the RFS algorithm allows the attacker to configure the length of history to make steadiness decisions, which enables the attacker to select between a fast and slow transition between attacks. A smaller history window would gradually adapt to the new scenario while a larger history window would slowly adapt to ensure a graceful increase in the number of attacks launched.

Figure 17 illustrates different scenarios and the attacks we launch under each scenario. In the first segment partner avatar is helping user jog at a fixed speed and the attacker wants to increase the jogging speed of the user. To achieve that, she launches a *slowdown* attack to give the application an impression that the user is walking slowly and the application signals the user to increase her speed, achieving the attacker's goal. In the second segment, the user wants to walk in a straight line while reducing her speed to rest her body and the attacker wants to deviate the user from the straight line. To do that, the attacker launches a *zero displacement* attack, which results in the fluctuations in the user's speed while also deviating her from the straight path.

In the third segment, the user is again trying to jog at a fixed speed and the attacker wants to slow her down. To achieve that, the attacker launches a *speedup* attack giving the app an impression that the user is speeding up. The feedback loop of the application then instructs the user to reduce her speed, which achieves the attacker's goal. In the final segment, the user is trying to increase her speed and finish at her desired end point. However, the attacker decides to launch the *path deviation* attack and deviate the user from her destination. As we see in the figure, the application perceives that the user is maintaining a fixed speed and instructs her to jog faster. The increase in speed, combined with the shift in deviation, results in the user losing the destination.

6.7 Visualizing Real-World Impact

Previously we have demonstrated configurability, efficacy, stealthiness, and robustness of our attack surface on partner avatar jogging scenario with the real world. In this section we are visualizing the impact and efficiency of our attack by demonstrating zero-displacement attack carried out on user trajectory while the user is jogging with avatar on local hiking trail surrounded by a pond (water body) and 13 acres conservation area. In Figure 18 both the original trajectory of the user (black line) and trajectory after the attack (red line) can be seen. To reiterate, the goal of zero-displacement attack is to diverge users such that it reaches the same destination but through a different path. It is intuitive to see that the path covered by the user in the attack trajectory (red line) is

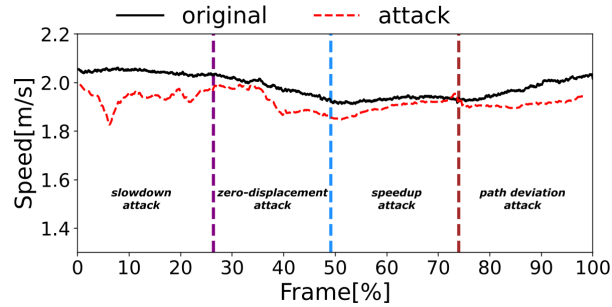


Fig. 17. Our attacks rely only on local information and allow attackers to change attacks based on the context.

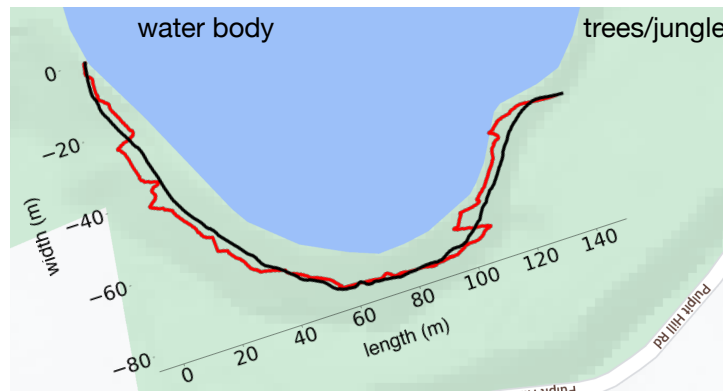


Fig. 18. Real world attack illustration.

different from the original (black line). However, it is interesting to note that when the path is relatively straight, the divergence is slightly higher than when the path is curved or changing. This highlights a key point that our approach is efficient enough to be aware of users' surroundings. While our algorithm is not employing any tracking technique to know the details of surrounding, its reliance on steadiness incorporates that information implicitly. Lastly, even though divergence is smaller on curvy paths (frequent change in trajectory) for ensuring stealthiness, there is still some divergence that can be attained by an attacker with our approach. This also highlights that our attack surface is not only proven to be stealthy and practical for this particular use-case scenario, but it shows the potential to be damaging in other critical scenarios where the users' surroundings is not changing too drastically such as surgery [28] and factory floors [57].

6.8 Microbenchmarks

We also present benchmarking results to demonstrate that our attacks are fast and simple, which allows them to be used in a fast-paced environment of MR. For a successful and stealthy launch, the attacker needs to decide on launching the attack, select an attack type, compute similarity index, and perform frame manipulations. In our use case, the visual stream has 5-30 fps rate, while inertial sensors have 12-20 fps. As a result, our attack should complete in less than 33.33ms, based on the attack vector with the highest sampling rate. We empirically calculated the time it takes to complete all of the different attack steps. The computation time of similarity index for the inertial vector on average is 0.8ms (1.5ms max and 0.7ms min). For visual attack vectors, average computation time is 10ms (13.8ms max and 9.2ms min). The frame manipulation step takes 5ms on average. Finally, the RFS algorithm runs in 0.0011 ms. The total time is less than 24ms on average, which is less than the highest sampling rate making our attacks fast and stealthy.

7 DISCUSSION

In this section, we discuss possible remediation techniques for our proposed attack surface and how most of these techniques will be ineffective in detecting and mitigating the proposed attacks. Additionally, we discuss how the proposed attacks are not only applicable to MR but also apply to other relevant domains.

Potential Defense Mechanisms. For the inertial sensors, prior work has proposed data production level defenses such as acoustic dampening [14] that help reduce resonant interference in the inertial signal. Others have proposed better filtering approaches to mitigate spoofing of inertial sensors [67]. For the visual stream, visual challenges can be an effective tool to detect camera attacks [71] and watermarks can be added to images to protect their integrity. However, these preventive strategies require either a fundamental shift in sensor hardware design or heavily instrumenting the environment, both are not feasible for practical and large-scale deployments. Also, our work shows that deep neural networks are ineffective in neutralizing concurrent manipulations across sensors and start adapting to malicious inputs induced by the attacker.

Other techniques include recovery methods like semantic analysis for capturing context information to evade the attacks. However, these techniques need a huge amount of contextual data labels along with pose data that could be used to capture the abnormal data using statistical characteristics with semantic and context information [61]. Data authentication using cryptography [23] mechanism at sensor fusion level is resource-intensive for edge devices. Finally, online intrusion detection [29] using cryptography mechanisms like entropy, frame intervals, frame consistency and correlation, data fingerprinting can be used to detect attacked data frames. The solution presented by these techniques is to drop the attack frames at the tracking algorithm. If an attacker learns this behavior, the attacker might use this behavior against the tracking algorithm to cause large frame drops to induce further attacks.

Broad Applicability. Our approach has broader impacts both at different tracking scales and across different domains. For instance, our tracking use cases in navigation and fitness have movements that stretch from a few inches to meters. Our attack is also applicable to surgical and other use cases with micro-level movements. Also, our attack surface is transferable to other tracking-based systems such as autonomous vehicles [76], smart robots [85], and drones [19].

Attack Improvements To achieve the goal of exploring unknown attack surfaces in MR, we presented a proof-of-concept of our attacks on the real world data. Currently we presented our attack in terms of only two attack vectors, but our approach is not limited to only visual and inertial sensors. Any sensor modality that can aid in tracking can pose a threat to be utilized as an attack vector. In spatially rich environments such as MR, audio sensor can be a potential attack vector to estimate and/or misuse the users' where-about using sound waves. Additionally, our attack surface is based on black-box model, so attacker have no idea about the underlying tracking mechanisms, which makes it harder for attacker to launch these attacks, but once attackers get access to the tracking mechanisms these attacks will become more powerful in-terms of potential to cause damage. In future we plan to explore white-box model on our attack surface.

8 CONCLUSION AND FUTURE WORK

In this paper, we presented a novel multi-modal spatiotemporal attack surface for MR systems. Our proposed attacks concurrently launch attacks on multiple sensing streams to achieve *fast*, *stealthy*, and *precise* outcomes. We proposed a generic framework to launch attacks that, in the case of a tracking application, can cause a user to slow down, speed up, deviate from the intended path, and lose the destination. We evaluated the efficacy of our proposed attacks against the state-of-the-art sensor fusion-based tracking algorithm and demonstrated that the attacker is able to achieve the goal across a wide range of environments and user action scenarios. In future we plan to explore machine learning techniques for finding the right time to attack, and context-aware optimization techniques for maximizing the attack impact and to keep it stealthy. We also aim to devise multimodal learning mechanisms to neutralize the proposed attack surface.

REFERENCES

- [1] 2020. Hololens 2. <https://www.microsoft.com/en-us/hololens/>.
- [2] 2020. The ghost pacer. <https://ghostpacer.com/>.
- [3] 2021. Lynx R1 Headset. <https://www.lynx-r.com/products/lynx-r1-headset>.
- [4] Roghieh Abdollahi Biroon, Pierluigi Pisu, and Zoleikha Abdollahi Biron. 2020. Real-time False Data Injection Attack Detection in Connected Vehicle Systems with PDE modeling. In *IEEE ACC*.
- [5] Suhas G Aekanth. 2023. Transforming E-Learning Through the Use of Virtual and Augmented Reality: A Systematic Review. *Human-Automation Interaction*.
- [6] Jad Al Rabbaa, Alexis Morris, and Sowmya Somanath. 2019. MRsive: An Augmented Reality Tool for Enhancing Wayfinding and Engagement with Art in Museums. In *HCI International-Posters*, Constantine Stephanidis (Ed.).
- [7] Sean Andrist, Dan Bohus, Ashley Feniello, and Nick Saw. 2022. Developing Mixed Reality Applications with Platform for Situated Intelligence. *IEEE VRW*.
- [8] Fatima M Anwar, Luis Garcia, Xi Han, and Mani Srivastava. 2019. Securing Time in Untrusted Operating Systems with TimeSeal. *IEEE RTSS*.
- [9] Fatima M Anwar and Mani Srivastava. 2020. A Case for Feedforward Control with Feedback Trim to Mitigate Time Transfer Attacks. *ACM TOPS*.
- [10] Connor Bolton, Sara Rampazzi, Chaohao Li, Andrew Kwong, Wenyuan Xu, and Kevin Fu. 2018. Blue note: How intentional acoustic interference damages availability and integrity in hard disk drives and operating systems. In *2018 IEEE S&P*.
- [11] Daniel Boulosa, Jonathan Esteve-Lanao, Arturo Casado, Leonardo A Peyré-Tartaruga, Rodrigo Gomes da Rosa, and Juan Del Coso. 2020. Factors affecting training and physical performance in recreational endurance runners. *MDPI Sports* 8 (2020).
- [12] Elisabeth Broneder, Christoph Weiß, Julian Thöndel, Emanuel Sandner, Stephanie Puck, Monika Puck, Gustavo Fernández Domínguez, and Miroslav Sili. 2022. TACTILE– Mixed Reality-Based System for Cognitive and Physical Training. In *IHIET-FS*.
- [13] Peter Casey, Ibrahim Baggili, and Ananya Yarramreddy. 2019. Immersive Virtual Reality Attacks and the Human Joystick. *IEEE TDSC*.
- [14] Simon Castro, Robert Dean, Grant Roth, George T Flowers, and Brian Grantham. 2007. Influence of Acoustic Noise on the Dynamic Performance of MEMS Gyroscopes. *ASME IMECE*.
- [15] Changhao Chen, Stefano Rosa, Yishu Miao, Chris Xiaoxuan Lu, Wei Wu, Andrew Markham, and Niki Trigoni. 2019. Selective sensor fusion for neural visual-inertial odometry. *IEEE/CVF CVPR*.
- [16] Ronald Clark, Sen Wang, Hongkai Wen, A. Markham, and Agathoniki Trigoni. 2017. VINet: Visual-Inertial Odometry as a Sequence-to-Sequence Learning Problem. *AAAI*.
- [17] Patel Dhruv and Subham Naskar. 2020. Image classification using convolutional neural network (CNN) and recurrent neural network (RNN): a review. *Springer ICMLIP* (2020).
- [18] Michalis Diamantaris, Serafeim Moustakas, Lichao Sun, Sotiris Ioannidis, and Jason Polakis. 2021. This Sneaky Piggy Went to the Android Ad Market: Misusing Mobile Sensors for Stealthy Data Exfiltration. In *ACM CCS*.
- [19] Louis Dressel and Mykel J. Kochenderfer. 2019. Hunting Drones with Other Drones: Tracking a Moving Radio Target. *ICRA*.
- [20] Jared R Fletcher and Brian R MacIntosh. 2017. Running economy from a muscle energetics perspective. In *Frontiers in Physiology*, Vol. 8.
- [21] Michela Franzò, Andrada Pica, Simona Pascucci, Mariano Serrao, Franco Marinozzi, and Fabiano Bini. 2023. A Proof of Concept Combined Using Mixed Reality for Personalized Neurorehabilitation of Cerebellar Ataxic Patients. In *MDPI Sensors*, Vol. 23.
- [22] Chenglong Fu, Qiang Zeng, and Xiaojiang Du. 2021. Hawatcher: Semantics-aware anomaly detection for appified smart homes. In *USENIX Security*.
- [23] Ethan Gaebel, Ning Zhang, Wenjing Lou, and Y. Thomas Hou. 2016. Looks Good To Me: Authentication for Augmented Reality. *ACM TrustED*.
- [24] Filippas Giannakas, Christos Troussas, Akrivi Krouska, Ioannis Voyiatzis, and Cleo Sgouropoulou. 2022. Blending cybersecurity education with IoT devices: A u-Learning scenario for introducing the man-in-the-middle attack. In *Information Security Journal: A Global Perspective*. Taylor & Francis.
- [25] Ilias Giechaskiel and Kasper Rasmussen. 2019. Taxonomy and challenges of out-of-band signal injection attacks and defenses. In *IEEE Communications Surveys & Tutorials*, Vol. 22.
- [26] Águeda Gómez-Cambronero, Alberto González-Pérez, Ignacio Miralles, and Sven Casteleyn. 2019. Mixed Reality Escape Room Video Game to Promote Learning In Indoor Environments. *EDULEARN19*.
- [27] Cassandra Gorman and Louise Gustafsson. 2022. The Use of Augmented Reality for Rehabilitation After Stroke: A Narrative Review. *DRAT*.
- [28] Thomas M Gregory, Jules Gregory, John Sledge, Romain Allard, and Olivier Mir. 2018. Surgery guided by mixed reality: presentation of a proof of concept. *Acta orthopaedica*.
- [29] Bogdan Groza and Pal-Stefan Murvay. 2019. Efficient Intrusion Detection With Bloom Filtering in Controller Area Networks. *IEEE TIFS*.

- [30] Takeo Hamada, Ari Hautasaari, Michiteru Kitazaki, and Noboru Koshizuka. 2022. Solitary Jogging with A Virtual Runner using Smartglasses. In *IEEE VR*.
- [31] Qingying Hao, Licheng Luo, Steve T.K. Jan, and Gang Wang. 2021. It's Not What It Looks Like: Manipulating Perceptual Hashing Based Applications. *ACM CCS*.
- [32] Jassim Happa, Mashhuda Glencross, and Anthony Steed. 2019. Cyber Security Threats and Challenges in Collaborative Mixed-Reality. *Frontiers in ICT*.
- [33] Zhitao He and Thiemo Voigt. 2013. Droplet: A New Denial-of-Service Attack on Low Power Wireless Sensor Networks. *IEEE MASS*.
- [34] Christos Ioannou, Patrick Archard, Eamonn O'Neill, and Christof Lutteroth. 2019. Virtual Performance Augmentation in an Immersive Jump & Run Exergame. In *ACM CHI*.
- [35] Suman Jana, David Molnar, Alexander Moshchuk, Alan Dunn, Benjamin Livshits, Helen J Wang, and Eyal Ofek. 2013. Enabling fine-grained permissions for augmented reality applications with recognizers. In *USENIX Security*.
- [36] Nicolas Kakouris, Numan Yener, and Daniel T.P. Fong. 2021. A systematic review of running-related musculoskeletal injuries in runners. *Journal of SHS*.
- [37] Anunay Kulshrestha. 2012. On the Hamming Distance between base-n representations of whole numbers. *ArXiv*.
- [38] Denis Foo Kune, John Backes, Shane S Clark, Daniel Kramer, Matthew Reynolds, Kevin Fu, Yongdae Kim, and Wenyan Xu. 2013. Ghost talk: Mitigating EMI signal injection attacks against analog sensors. In *IEEE S & P*.
- [39] Kiron Lebeck, Tadayoshi Kohno, and Franziska Roesner. 2016. How to Safely Augment Reality: Challenges and Directions. *ACM HotMobile*.
- [40] Kiron Lebeck, Tadayoshi Kohno, and Franziska Roesner. 2019. Enabling Multiple Applications to Simultaneously Augment Reality: Challenges and Directions. *ACM HotMobile*.
- [41] Bochao Li, Anthony Nordman, James Walker, and Scott A Kuhl. 2016. The effects of artificially reduced field of view and peripheral frame stimulation on distance judgments in HMDs. In *ACM Applied Perception*.
- [42] Mingyang Li and Anastasios Mourikis. 2013. High-precision, consistent EKF-based visual-inertial odometry. *Journal of Robotics Research*.
- [43] Yan Li, Yao Cheng, Weizhi Meng, Yingjiu Li, and Robert H. Deng. 2021. Designing Leakage-Resilient Password Entry on Head-Mounted Smart Wearable Glass Devices. In *IEEE TIFS*, Vol. 16.
- [44] Z. Li, Q. Yue, C. Sano, W. Yu, and X. Fu. 2017. 3D vision attack against authentication. In *IEEE ICC*.
- [45] Yonggen Ling, Linchao Bao, Zequn Jie, Fengming Zhu, Ziyang Li, Shanmin Tang, Yongsheng Liu, W. Liu, and T. Zhang. 2018. Modeling Varying Camera-IMU Time Offset in Optimization-Based Visual-Inertial Odometry. *ECCV*.
- [46] Zhen Ling, Zupei Li, Chen Chen, Junzhou Luo, Wei Yu, and Xinwen Fu. 2019. I Know What You Enter on Gear VR. In *IEEE CNS*.
- [47] Fangfang Lu, Hao Zhou, Lingling Guo, Jingjing Chen, and Licheng Pei. 2021. An ARCore-Based Augmented Reality Campus Navigation System. *Applied Sciences* 11.
- [48] Shiqing Luo, Xinyu Hu, and Zhisheng Yan. 2022. HoloLogger: Keystroke Inference on Mixed Reality Head Mounted Displays. In *IEEE VR*.
- [49] Frank J Massey Jr. 1951. The Kolmogorov-Smirnov test for goodness of fit. *ASA*.
- [50] Lorena CS Moreira, Regina C Ruschel, and Amir H Behzadan. 2023. Augmented Reality for Building Maintenance and Operation. In *Springer Handbook of Augmented Reality*. Springer, 495–532.
- [51] Abhishek Mukhopadhyay, Vinay Krishna Sharma, Prashant Gaikwad Tatyrao, Aumkar Kishore Shah, Ananthram MC Rao, P Raj Subin, and Pradipta Biswas. 2023. A comparison study between XR interfaces for driver assistance in take over request. In *Transportation Engineering*, Vol. 11.
- [52] Deeraj Nagothu, Yu Chen, Erik Blasch, Alexander Aved, and Sencun Zhu. 2019. Detecting Malicious False Frame Injection Attacks on Surveillance Systems at the Edge Using Electrical Network Frequency Signals. *Sensors*.
- [53] Youngseok Park, Yunmok Son, Hocheol Shin, Dohyun Kim, and Yongdae Kim. 2016. This ain't your dose: Sensor spoofing attack on medical infusion pump. In *USENIX Workshop on Offensive Technologies*.
- [54] Xukan Ran, Carter Slocum, Maria Gorlatova, and Jiasi Chen. 2019. ShareAR: Communication-Efficient Multi-User Mobile Augmented Reality. *ACM HotNets*.
- [55] Somaieh Rokhsaritalemi, Abolghasem Sadeghi-Niaraki, and Soo-Mi Choi. 2020. A review on mixed reality: Current trends, challenges and prospects. In *Applied Sciences*, Vol. 10. MDPI.
- [56] Priyanka Samanta and Shweta Jain. 2021. Analysis of Perceptual Hashing Algorithms in Image Manipulation Detection. *Procedia CS*.
- [57] Bernd Schwald and Blandine De Laval. 2003. An augmented reality system for training and assistance to maintenance in the industrial context. UNION Agency-Science Press.
- [58] Thomas Schöps, Torsten Sattler, and Marc Pollefeys. 2019. BAD SLAM: Bundle Adjusted Direct RGB-D SLAM. *IEEE/CVF CVPR*.
- [59] Jayaprakash Selvaraj, Gökçen Yılmaz Dayanıklı, Neelam Prabhu Gaunkar, David Ware, Ryan M Gerdes, and Mani Mina. 2018. Electro-magnetic induction attacks against embedded systems. In *Asia CCS*.

- [60] R. A. Sharma, A. Dongare, J. Miller, N. Wilkerson, D. Cohen, V. Sekar, P. Dutta, and A. Rowe. 2020. All that GLITTERs: Low-Power Spoof-Resilient Optical Markers for Augmented Reality. *ACM IPSN*.
- [61] Amit Kumar Sikder, Hidayet Aksu, and A. Selcuk Uluagac. 2017. 6thSense: A Context-aware Sensor-based Attack Detector for Smart Devices. *USENIX Security*.
- [62] Thibault Simonetto, Salijona Dyrnishi, Salah Ghamizi, Maxime Cordy, and Yves Le Traon. 2021. A unified framework for adversarial attack and defense in constrained feature space. *ArXiv preprint arXiv:2112.01156* (2021).
- [63] Vivek Singh, Pallav Pant, and Ramesh Tripathi. 2015. Detection of Frame Duplication Type of Forgery in Digital Video Using Sub-block Based Features. In *ICDF2C*.
- [64] Ivo Sluganovic, Mihael Liskij, Ante Derek, and Ivan Martinovic. 2020. Tap-pair: Using spatial secrets for single-tap device pairing of augmented reality headsets. In *ACM CDASP*.
- [65] I. Sluganovic, M. Serbec, A. Derek, and I. Martinovic. 2017. HoloPair: Securing Shared Augmented Reality Using Microsoft HoloLens. *ACSAC*.
- [66] Yunmok Son, Hocheol Shin, Dongkwan Kim, Youngseok Park, Juhwan Noh, Kibum Choi, Jungwoo Choi, and Yongdae Kim. 2015. Rocking drones with intentional sound noise on gyroscopic sensors. In *USENIX Security*.
- [67] Timothy Trippel, Ofir Weisse, Wenyuan Xu, Peter Honeyman, and Kevin Fu. 2017. WALNUT: Waging Doubt on the Integrity of MEMS Accelerometers with Acoustic Injection Attacks. *IEEE EuroS&P*.
- [68] Mariapina Trunfio, Timothy Jung, and Salvatore Campana. 2022. Mixed reality experiences in museums: Exploring the impact of functional elements of the devices on visitors' immersive experiences and post-experience behaviours. In *Information & Management*, Vol. 59. Elsevier.
- [69] Y. Tu, Z. Lin, I. Lee, and X. Hei. 2018. Injected and Delivered: Fabricating Implicit Control over Actuation Systems by Spoofing Inertial Sensors. *USENIX Security*.
- [70] Dorin Ungureanu, Federica Bogo, S. Galliani, Pooja Sama, Xin Duan, Casey Meekhof, Jan Stühmer, Thomas J. Cashman, Bugra Tekin, Johannes L. Schönberger, Pawel Olszta, and Marc Pollefeys. 2020. HoloLens 2 Research Mode as a Tool for Computer Vision Research. *ArXiv*.
- [71] J. Valente, K. Bahirat, K. Venechanos, A.A. Cardenas, and P. Balakrishnan. 2019. Improving the Security of Visual Challenges. *ACM TCPS*.
- [72] Junia Valente and Alvaro A. Cardenas. 2017. Remote Proofs of Video Freshness for Public Spaces. In *ACM CPS (Workshop)*. Association for Computing Machinery.
- [73] Mathy Vanhoef. 2021. Fragment and Forge: Breaking Wi-Fi Through Frame Aggregation and Fragmentation. *USENIX Security*.
- [74] Brandon Wagstaff and Jonathan Kelly. 2018. LSTM-based zero-velocity detection for robust inertial navigation. In *IEEE IPIN*.
- [75] He Wang, Ted Tsung-Te Lai, and Romit Roy Choudhury. 2015. MoLe: Motion Leaks through Smartwatch Sensors. In *MobiCom*.
- [76] Heng Wang and Xiaodong Zhang. 2022. Real-time vehicle detection and tracking using 3D LiDAR. *Asian Journal of Control*.
- [77] Chris Warin and Delphine Reinhardt. 2022. Vision: Usable Privacy for XR in the Era of the Metaverse. *EuroUSEC*.
- [78] Carolin Wienrich, Philipp Komma, Stephanie Vogt, and Marc E. Latoschik. 2021. Spatial Presence in Mixed Realities—Considerations About the Concept, Measures, Design, and Experiments. *Frontiers in VR*.
- [79] Fridolin Wild, Lawrence Marshall, Jay Bernard, Eric White, and John Twycross. 2021. UNBODY: A Poetry Escape Room in Augmented Reality. *Information 12*.
- [80] Robert Xiao and Hrvoje Benko. 2016. Augmenting the field-of-view of head-mounted displays with sparse peripheral displays. In *ACM CHI*.
- [81] Yi Xu, Yuzhang Wu, and Hui Zhou. 2018. Multi-Scale Voxel Hashing and Efficient 3D Representation for Mobile Augmented Reality. *IEEE CVPR Workshops*.
- [82] Chandra Shekhar Yadav, Jagendra Singh, Aruna Yadav, Himansu Sekhar Pattanayak, Ravindra Kumar, Arfat Ahmad Khan, Mohd Anul Haq, Ahmed Alhussen, and Sultan Alharby. 2022. Malware Analysis in IoT & Android Systems with Defensive Mechanism. In *Electronics*, Vol. 11. MDPI.
- [83] Sungjae Yoo and Jiro Tanaka. 2022. Increasing Motivation of Walking Exercise Using 3D Personalized Avatar in Augmented Reality. In *HCI 2022*.
- [84] Guoming Zhang, Chen Yan, Xiaoyu Ji, Tianchen Zhang, Taimin Zhang, and Wenyuan Xu. 2007. DolphinAttack: Inaudible Voice Commands. In *ACM CCS*.
- [85] Hui Zhang, Li Zhu Liu, He Xie, Yiming Jiang, Jian Zhou, and Yaonan Wang. 2022. Deep Learning-Based Robot Vision: High-End Tools for Smart Manufacturing. *IEEE IMM*.
- [86] Shijia Zhang, Yilin Liu, and Mahanth Gowda. 2023. I Spy You: Eavesdropping Continuous Speech on Smartphones via Motion Sensors. In *ACM IMWUT*, Vol. 6.
- [87] Jianyu Zhao, Ergo Pikas, Olli Seppänen, and Antti Peltokorpi. 2021. Using Real-Time Indoor Resource Positioning to Track the Progress of Tasks in Construction Sites.

- [88] Xiaoming Zhao, Jingmeng Liu, Xingming Wu, Weihai Chen, Fanghong Guo, and Zhengguo Li. 2022. Probabilistic Spatial Distribution Prior Based Attentional Keypoints Matching Network. *IEEE TCSVT*.
- [89] Fengyuan Zhu and Tovi Grossman. 2020. BISHARE: Exploring Bidirectional Interactions Between Smartphones and Head-Mounted Augmented Reality. *ACM CHI*.